

ALISSON GABRIEL DOS SANTOS ALVES

Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada

Salvador

2026

ALISSON GABRIEL DOS SANTOS ALVES

Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada

Trabalho de Conclusão de Curso apresentado ao Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, como parte dos requisitos para obtenção do Certificado de conclusão de curso. **Curso:** Técnico em

Informática **Orientador:** Gilberto Monteiro

Salvador Ano 2026

Resumo

ALVES, Alisson Gabriel dos Santos. **Inkdroid: Repurposing de tablets legados via debloat automatizado para leitura focada**. 2026. Trabalho de Conclusão de Curso (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

Analisa o impacto do padrão de consumo linear na indústria de hardware mobile e o fenômeno do envelhecimento de software (*software aging*), que gera descartes precoces de dispositivos funcionalmente íntegros. Define como objetivo central o desenvolvimento de uma solução de software para promover a preservação ambiental e a saúde digital por meio da ressignificação (*repurposing*) de tablets Android legados em dispositivos de leitura focada (E-readers). Utiliza como objeto de estudo o tablet Multilaser M8 4G da rede pública estadual da Bahia. Emprega uma metodologia aplicada e experimental, fundamentada no uso da ferramenta Android Debug Bridge (ADB) para intervenções não invasivas em três camadas sistêmicas: aplicação, framework e kernel. O método compreende a execução de um *debloat* radical para remoção de serviços redundantes, o ajuste de parâmetros ergonômicos como escala de cinza e densidade de pixels para mitigar a Síndrome da Visão do Computador (CVS), além da aplicação de *underclock* e redução da taxa de atualização da tela para otimização energética. Valida a eficácia da intervenção por meio de análise quantitativa de desempenho e autonomia, utilizando métricas de *Throughput*, *Slowdown* e o modelo de Desejabilidade Composta para equilibrar performance e consumo. Conclui que a engenharia de software consciente permite a conversão de potenciais resíduos eletrônicos em ativos educacionais valiosos, comprovando a viabilidade técnica de políticas de TI sustentáveis e a democratização do acesso a ferramentas de leitura fluida e livre de distrações.

Palavras-chave: Debloat Android. Green IT. Saúde Digital. Obsolescência de Software. Deep Work.

Abstract

ALVES, Alisson Gabriel dos Santos. **Inkdroid: Repurposing legacy tablets via automated debloat for focused reading**. 2026. Graduation Thesis (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

This research analyzes the impact of linear consumption patterns in the mobile hardware industry and the phenomenon of software aging, which triggers the premature disposal of functionally sound devices. The central objective is the development of a software solution that combines environmental preservation and digital health by repurposing legacy Android tablets into focused reading devices (E-readers). The study specifically focuses on the Multilaser M8 4G tablet distributed within the Bahia state public school system. The methodology is classified as applied and experimental, utilizing a mixed approach to validate hardware repurposing. The technical procedure employs the Android Debug Bridge (ADB) for non-invasive interventions across three systemic layers: application, framework, and kernel. The method involves automated debloating to remove redundant services and Google Mobile Services (GMS), the adjustment of ergonomic parameters such as grayscale and pixel density to mitigate Computer Vision Syndrome (CVS), and kernel-level underclocking to maximize energy efficiency. The effectiveness of the intervention is validated through quantitative performance and autonomy analysis, using Throughput and Slowdown metrics alongside Composite Desirability models to balance performance and power consumption. It concludes that conscious software engineering enables the conversion of potential electronic waste into valuable educational assets, proving the technical viability of sustainable IT policies and the democratization of distraction-free reading tools.

Keywords: Android Debloat. Green IT. Digital Health. Software Obsolescence. Deep Work.

Sumário

1	GLOSSÁRIO	6
2	INTRODUÇÃO	9
2.1	Problema de Pesquisa	9
2.2	Objetivos	9
3	JUSTIFICATIVA	11
4	METODOLOGIA	12
5	DESENVOLVIMENTO E IMPLEMENTAÇÃO	14
5.1	Intervenções em Nível de Aplicação	14
5.1.1	Configuração do Ambiente e Protocolo de Comunicação	14
5.1.2	Lógica de Automação e Processamento de Bloatwares	15
5.2	Intervenção em Nível de Framework	16
5.2.1	Conceituação e Arquitetura de Framework	17
5.2.2	Fluxo de Execução e Pipeline	17
5.2.3	Estratégia de Modificação não Invasiva	19
5.2.4	Display e Síndrome da Visão do Computador	19
5.3	Bateria, Autonomia e Foco	21
5.4	Lógica de Programação, Arquitetura e Ferramentas do InkDroid	23
5.4.1	Evolução do Script, Modularidade e Modos de Execução	23
5.4.2	Verificação de Segurança e Preparação do Ambiente	23
5.4.3	Inicialização de Logs, Leitura da Blacklist e Debloat Automático	24
5.4.4	Download, Instalação de Aplicações e Definição do Launcher Padrão	24
5.4.5	Otimizações Ergonômicas do Display via Código	24
5.4.6	Ajustes Sistemáticos de Bateria e Funcionalidades da CLI	25
5.4.7	Arquitetura de Reversibilidade (InkDroid_uninstall.sh)	25
5.4.8	Necessidade do Refatoramento de Rede	26
5.4.9	Integração Híbrida Bash-Python	28
6	RESULTADOS	29
7	CONSIDERAÇÕES FINAIS	30
8	AGRADECIMENTOS	31

REFERÊNCIAS BIBLIOGRÁFICAS 32

1 GLOSSÁRIO

ADB (*Android Debug Bridge*)

Ferramenta de linha de comando oficial do AOSP que opera em modelo cliente-servidor, permitindo a comunicação, envio de instruções e automação de textitscripts entre a estação de trabalho e o dispositivo Android.

AOSP (*Android Open Source Project*)

Projeto de código aberto mantido pela Google que disponibiliza a estrutura base e a arquitetura do sistema operacional Android.

API (*Application Programming Interface*)

Interface de Programação de Aplicações; conjunto de rotinas e padrões de software que permite a comunicação entre diferentes componentes de sistema.

ART (*Android Runtime*)

Ambiente de execução do Android responsável por traduzir o código das aplicações em instruções de máquina executadas pelo processador.

Blacklist

Arquivo de configuração estruturado (em formato JSON) utilizado para catalogar e especificar os identificadores de pacotes (*package names*) alvos de remoção.

Bloatware

Softwares e aplicativos pré-instalados por fabricantes ou operadoras que operam em segundo plano e consomem memória e ciclos de processamento desnecessários.

CLI (*Command Line Interface*)

Interface de linha de comando utilizada para a execução de programas e *scripts* em modo texto no terminal.

CPU (*Central Processing Unit*)

Unidade Central de Processamento; componente de hardware responsável pela execução dos cálculos e instruções do sistema.

CVS (*Computer Vision Syndrome*)

Síndrome da Visão do Computador; condição ocupacional caracterizada por fadiga e desconforto ocular resultantes do uso prolongado de telas e terminais de vídeo.

Daemon	Processo do sistema operacional que executa de forma contínua em segundo plano sem a necessidade de interatividade direta com o usuário (ex.: <i>adbd</i>).
Debloat	Processo sistemático de limpeza e desinstalação de pacotes, aplicativos e serviços redundantes de um sistema operacional.
Deep Work	Trabalho profundo; conceito que se refere ao estado de concentração contínua e redução de estímulos/distrações induzido por dispositivos de propósito único.
Desejabilidade Composta	Modelo de análise estatística e multicritério utilizado para calcular o ponto de equilíbrio ótimo entre métricas de desempenho (<i>throughput</i>) e consumo de energia.
Dispositivo Legado (<i>Legacy Device</i>)	Hardware perfeitamente funcional que se tornou obsoleto em relação às exigências de softwares e atualizações recentes.
DPI (<i>Dots Per Inch</i>)	Densidade de pontos por polegada; métrica utilizada para ajustar a escala e a proporção dos elementos visuais da interface do Android.
E-ink / E-reader	Tecnologia de tela de tinta eletrônica reflexiva e dispositivos leitores de livros digitais projetados para alta ergonomia visual.
E-waste	Resíduos de equipamentos eletroeletrônicos acumulados devido ao descarte e à obsolescência de hardware.
GMS (<i>Google Mobile Services</i>)	Conjunto de aplicativos, telemetrias e serviços proprietários da Google que executam em segundo plano no sistema Android.
HAL (<i>Hardware Abstraction Layer</i>)	Camada de Abstração de Hardware que faz a ponte entre o <i>framework</i> do Android e os <i>drivers</i> do Kernel Linux.
IPC (<i>Inter-Process Communication</i>)	Comunicação entre processos; mecanismo que permite a troca segura de dados e chamadas entre diferentes processos no sistema (como a arquitetura Binder).

JSON (*JavaScript Object Notation*)

Formato leve e estruturado para troca e armazenamento de dados em pares de chave-valor.

Kernel

Núcleo do sistema operacional (baseado em Linux no Android) responsável pela alocação de memória, escalonamento e comunicação com o hardware.

Launcher

Aplicativo gerenciador da tela de início e da organização de ícones e interfaces gráficas do Android.

Minimalismo Digital

Filosofia de otimização do uso de ferramentas digitais para reduzir a fragmentação da atenção e focar em tarefas prioritárias.

RAM (*Random Access Memory*)

Memória de acesso aleatório onde são armazenados os dados temporários dos processos em execução.

Settings.db

Banco de dados interno do Android responsável por armazenar as configurações globais e comportamentos de sistema.

Shell

Interpretador de comandos do sistema operacional Unix/Linux responsável por receber e executar comandos de terminal.

Slowdown

Métrica de degradação de desempenho que avalia o atraso relativo de um processo sob condições de carga do sistema.

Software Aging

Envelhecimento de software; conceito que explica a perda progressiva de fluidez em sistemas motivada pelo acúmulo de serviços e atualizações pesadas em hardwares antigos.

SurfaceFlinger

Serviço do Android responsável por compor as superfícies visuais e controlar a taxa de atualização enviada ao display.

UI (*User Interface*)

Interface do usuário; conjunto de componentes visuais através dos quais o usuário interage com o sistema.

Underclocking

Redução proposital da frequência de operação do processador com o objetivo de diminuir o consumo de bateria e a dissipação térmica.

2 INTRODUÇÃO

A crise ambiental contemporânea é considerada por muitos o problema de maior relevância da atualidade, sendo essa, muito impulsionada pelo atual padrão de consumo linear da indústria no geral. Esse padrão se mostra ainda mais forte no contexto de *hardwares mobile*, que são renovados a cada ano, prometendo grandes mudanças, deixando em pouco tempo aparelhos funcionais obsoletos ou "artificialmente incapazes", Parnas (1994) diz que esse efeito está fundamentalmente ligado ao que ele chama de *software aging* onde a medida que as aplicações se atualizam elas exigem cada vez mais do hardware, ainda que o mesmo seja perfeitamente capaz tornando o software em um "fardo oneroso". Essa prática impulsiona o aumento da geração de resíduos sólidos, em especial, o lixo eletroeletrônico, que possuem metais pesados de difícil decomposição. Segundo o relatório The Global E-waste Monitor (BALDé et al., 2024), a geração de lixo eletrônico, estimada em 62 bilhões de kg no ano de 2022, cresce cinco vezes mais do que a reciclagem documentada, com apenas 22,3% dos resíduos coletados adequadamente em 2022.

2.1 Problema de Pesquisa

A "incapacidade artificial" nos dispositivos móveis é impulsionada pelo *overhead* de processamento do Google Mobile Services (GMS) e pela evolução das camadas de *runtime* (ART) e pelas atualizações do sistema que vêm perfeitamente otimizadas para o hardware de última geração, mas com grandes falhas de otimização para os mais antigos. Isso deixa dispositivos com hardware perfeitamente funcionais, freitados pelo software.

2.2 Objetivos

A presente pesquisa tem por objetivo o desenvolvimento de uma solução de software que combine preservação ambiental e saúde digital através da *repurposing* de tablets Android, em E-readers, dando um novo significado funcional para dispositivos obsoletos. Em específico o tablet Multilaser M8 4G distribuído pelas escolas da rede pública estadual da Bahia.

Nesse dispositivo, será realizado um *debloat* automatizado para otimizar o uso do armazenamento, memória RAM e ciclos de CPU, junto a implementação de uma interface monocromática e ajustes ergonômicos para redução de fadiga visual. Tal preocupação com a fadiga visual e ergonomia, vem do crescente índice de CVS (Síndrome da Visão do Computador), a qual atinge cerca de 70% da população mundial (Sá, 2016). Além disso, outra preocupação da presente pesquisa é a validação da autonomia energética, conforme

Monteiro et al. (2023) o perfil do usuário e serviços de fundo são os maiores vilões da bateria. A automatização também terá como meta o ajuste de parâmetros do Kernel para maximizar a autonomia energética. Essa automação será disponibilizada de forma Open Source em portais como Github de forma gratuita e em linguagens como Shell Script (Bash) e Python.

3 JUSTIFICATIVA

A necessidade dessa pesquisa é fortemente fundamentada na urgência de combater o modelo de consumo linear consolidado atualmente. Em 2022, a categoria de resíduos sólidos, que inclui aparelhos eletrônicos como celulares, tablets e laptops, chegou a acumular 5 bilhões de kg de lixo (BALDÉ et al., 2024). Entretanto, o destino da maioria dos dispositivos obsoletos, não é o descarte, mas sim a chamada hibernação doméstica. Estima-se que famílias da União Europeia possuam cerca de 74 itens eletrônicos em suas casas, dos quais, boa parcela está quebrada ou sem uso. No Brasil, a Associação Brasileira de Reciclagem de Eletroeletrônicos e Eletrodomésticos (ABREE, 2024) também sinaliza que cerca de 85% dos brasileiros possuem algum tipo de aparelho eletrônico parado em suas casas. O Brasil também é o maior gerador de *E-waste* da América Latina segundo a ABREE (2024). Esses dados representam um panorama preocupante em que por diversos fatores, dispositivos legado em seu pleno funcionamento são encontrados no seu estado de hibernação. A presente pesquisa pode ajudar a mitigar os danos dessa situação, dando um novo propósito para aqueles dispositivos que já não servem bem as exigências dos softwares e aplicativos modernos.

Somado a isso, o desenvolvimento da pesquisa também ancora-se nos conceitos de saúde digital e minimalismo para promover o *Deep Work* por meio de modificações em várias das partes que compõem o sistema Android. De acordo com Newport (2019), a tecnologia dos dias de hoje é projetada para capturar nossa atenção, o que, em dispositivos de múltiplas funções mostra-se um agente de fragmentação do foco, com diversos serviços disputando pelo seu olhar e atrapalhando o chamado *Deep Work*. A proposta de ressignificação visa também combater essa natureza de modo a tornar o aparelho em um dispositivo de leitura livre de distrações.

Finalmente, o projeto possui um forte componente de conveniência e democratização tecnológica. Ao transformar um dispositivo comum, como Multilaser M8 4G da rede estadual, em uma ferramenta educacional fluida, o estudo remove a barreira financeira imposta pelo alto custo de dispositivos especializados como o Kindle. Trata-se de uma prova de conceito de que a Engenharia Verde pode converter um dispositivo que seria trocado e descartado, em um ativo educacional valioso e acessível.

4 METODOLOGIA

A presente pesquisa classifica-se como aplicada e experimental, com abordagem mista para validar a eficácia da ressignificação do hardware. O procedimento técnico central vai utilizar o Android Debug Bridge (ADB), como principal ferramenta, permitindo a comunicação entre a automação e o tablet de forma nativa e segura. Trata-se de uma ferramenta de CLI (*Command Line Interface*) oficial de desenvolvimento disponibilizada pelo Android Open Source Project (AOSP). Operando no modelo cliente-servidor, ela dá acesso ao Shell Unix dos dispositivos Android de forma não invasiva e sem necessidade de acesso root (ANDROID DEVELOPERS, 2025). Essa ferramenta possibilita a escrita de um código para realizar alterações profundas nos dispositivos, essas que vão acontecer em 3 camadas principais:

1. **Nível de Aplicação:** Nesta etapa o script vai, de forma automática, instalar os aplicativos necessários para o pleno funcionamento do dispositivo como um E-reader, e remover processos redundantes ou aplicativos pré-instalados (*bloatwares*), fazendo um *debloat* completo. Desse modo, além de remover redes sociais, apps que prejudicam o *deep work* e atrapalham o foco, também serão removidos, caso o usuário optar, os chamados Google Mobile Services (GMS) que incluem telemetrias intrusivas. Esta ação libera recursos desperdiçados em coisas desnecessárias e constroem a base para as modificações mais profundas.
2. **Nível de Framework:** Essa é a fase em que ocorre a manipulação do banco de dados `textttsettings.db` e do gestor de janelas (*Window Manager* ou WM) para alterar diversas das configurações padrão. Nessa fase, algumas das principais modificações vão ser a aplicação de um filtro de luz azul nativo, simulando E-Inks de E-reader especializados. Essa alteração mostra-se relevante pois, segundo Benedetto et al. (2013), as telas E-ink diminuem drasticamente o risco de CSV, e embora a tela continue sendo LCD, a redução da luz azul e a escala de cinza reduzem o brilho e o contraste agressivo, mitigando a CVS, embora não atinja as propriedades físicas de reflexão de luz do papel eletrônico real aumentará o conforto ocular do usuário. A manipulação da densidade de pixels (DPI) e da escala de cinza também será aplicada visando otimizar a renderização de fontes, reduzindo a carga cognitiva e visual durante a leitura (Sá, 2016; ANDROID DEVELOPERS, 2025).
3. **Nível de Kernel:** Por fim, essa é a parte de maior dificuldade, mas que vai aplicar as maiores diferenças práticas. Aqui, a automação vai aplicar técnicas de *underclock* (limitação da frequência do processador) e redução da taxa de atualização da tela

via Surface Flinger, para tornar a tela do aparelho mais semelhante ao padrão da indústria, além de ajudar com a eficiência energética.

Essas modificações serão acompanhadas de uma análise de dados para medir a diferença de desempenho, uso de recursos de hardware e autonomia da bateria, por meio de uma comparação quantitativa usando métricas como *Throughput* e *Slowdown* para comprovar se a alteração foi de fato benéfica ou não do ponto de vista de fluidez do sistema, utilizando modelos de Desejabilidade Composta para equilibrar desempenho e consumo energético (PONTES, 2017).

5 Desenvolvimento e Implementação

O presente capítulo, descreve os procedimentos técnicos realizados para a elaboração da solução, estruturando-se conforme os níveis sistêmicos estabelecidos na metodologia: Aplicação, framework e Kernel. A implementação foi conduzida em ambiente desktop Linux, visando integração nativa com ferramentas de automação via terminal.

5.1 Intervenções em Nível de Aplicação

O primeiro nível de intervenção foca na desoneração do sistema através da remoção de processos e serviços irrelevantes para o produto final, bem como a integração das ferramentas necessárias para estabelecer uma experiência de leitura focada. Entretanto, antes de iniciar-se a etapa de desenvolvimento prático, faz-se necessário a preparação do ambiente e das ferramentas que vão permitir a comunicação entre os dispositivos.

5.1.1 Configuração do Ambiente e Protocolo de Comunicação

O desenvolvimento se inicia pela instalação e preparação da ferramenta *Android Debug Bridge* (ADB), que, em consonância com a metodologia estabelecida, trata-se de uma ferramenta de linha de comando (*Command Line Interface - CLI*) oficial do AOSP (ANDROID DEVELOPERS, 2025), que será responsável por cada um dos comandos e modificações realizadas ao longo do processo. A instalação do ADB, em especial em ambientes Linux desktop caracteriza-se pela simplicidade, se resumindo a um único comando do gerenciador de pacotes da distribuição. Contudo, apesar do rápido providenciamento, a ferramenta possui uma arquitetura robusta e abrangente.

Em termos funcionais, o ADB opera em uma arquitetura de cliente-servidor (TANENBAUM; BOS, 2016), o qual permite a troca de informações e o envio de instruções entre os dois dispositivos, nessa estrutura, o servidor (executado no tablet) atua de forma reativa, processando requisições e retornando dados, enquanto o cliente (executado na estação de trabalho) detém a prerrogativa de ordenar as execuções. Paralelamente, estabeleceu-se o controle de versão via repositório Git, permitindo o rastreamento de modificações e a organização colaborativa do código-fonte, parte crucial do projeto visto que o mesmo tem por objetivo se manter *open source*¹. A manutenção do código em formato aberto (*open source*) justifica-se pela necessidade de auditabilidade e transparência, sobretudo em um projeto que realiza modificações profundas no sistema operacional.

¹ Open Source é uma forma de manter projetos de software onde o autor disponibiliza todo seu código (source) publicamente para que outras pessoas ou empresas possam usar ou contribuir, desde que sigam as regras da sua licença

No entanto, por conceder acesso ao *shell Unix*² do celular, a abertura desse protocolo pode se tornar um vetor de invasão preocupante sem as devidas proteções. Para mitigar tais riscos, o sistema Android implementa medidas de segurança rigorosas fundamentadas em permissões de usuário, sandboxes e na autenticação por chaves de criptografia assimétrica RSA. No ecossistema Android cada um dos aplicativos roda em um ambiente isolado e tem um identificador de usuário único e exclusivo (UID) com permissões diferentes. Isso garante que, por padrão, nenhum programa tenha acesso a dados dos outros, nem ao sistema. Embora o Android, diferente de outros sistemas, não disponibilize um usuário root (administrador) por padrão, os comandos do ADB são executados por um usuário especial com permissões elevadas chamado "shell". Esse usuário pode gerenciar permissões de outros, ler *logs*³ de sistema, alterar configurações internas e desinstalar e instalar programas.

Para mitigar acessos não autorizados a esse usuário privilegiado, o protocolo utiliza de a criptografia assimétrica. Até a versão 4.2.2 do Android bastava ativar a depuração e qualquer máquina poderia executar comandos no *shell*, criando uma vulnerabilidade como a adulteração de aparelhos furtados, ou ataques por meio de estações de carregamento maliciosas (*juice jacking*) em locais públicos. Para corrigir isso o Google implementou uma autenticação por chave RSA, que funciona com uma chave pública (textttadbkey.pub) e uma privada (textttadbkey). Quando você conecta um computador, o celular pergunta se deve dar permissão para aquele computador, e, ao tocar em "Sempre permitir a partir desse computador", uma cópia da chave pública é salva no celular para validar as conexões futuras e garantir a integridade da automação sem violar os protocolos de segurança nativos do sistema.

5.1.2 Lógica de Automação e Processamento de Bloatwares

Consolidados o ambiente de desenvolvimento e repositório Git, o desenvolvimento da automação inicia-se através de uma verificação inteligente dos aparelhos servidores conectados. Essa medida visa assegurar que o *script* execute apenas quando houver exatamente um tablet conectado ao barramento USB, evitando possíveis conflitos entre os comandos, assim como previne que a automação rode sem nenhum dispositivo conectado.

Validada a conexão, a próxima fase seria a desoneração do sistema via remoção de softwares prescindíveis (debloat). A desinstalação de pacotes do dispositivo não se restringe a uma otimização superficial, mas constitui uma estratégia para conter o envelhecimento de software (software aging). Conforme fundamentado por Parnas (PARNAS, 1994), o envelhecimento de software ocorre quando o programa se torna um "fardo oneroso" devido

² Unix foi um dos primeiros e mais importantes sistemas operacionais, estabelecendo padrões usados até hoje inclusive no Android, Linux e MacOS.

³ Logs são registros do que acontece em tempo de execução de uma aplicação, usado para monitoramento e diagnósticos

ao acúmulo de atualizações e serviços contínuos, esses que são voltados para o hardware mais novo, mas também são empurrados para o dispositivo legado, demandando ainda mais recursos computacionais. Ao eliminar o excesso de processos, a automação restaura a fluidez do aparelho e diminui os agentes de distração, permitindo a execução de tarefas simples como a leitura, serem mais confortáveis e principalmente convenientes.

Para estruturar o *debloat* sem poluir o código principal com chamadas massantes, foi adotado uma abordagem declarativa baseada no arquivo de configuração `textttblacklist.json`. Esse arquivo lista os nomes exatos de pacotes de rede sociais, ferramentas do *Google Mobile Services* e demais *bloatwares* identificados como prejudiciais ao foco ou ao desempenho do sistema. A escolha da remoção do GMS fundamenta-se na eficiência energética: estudos de mapeamento sistemático indicam que aplicativos e serviços que operam em segundo plano são principais responsáveis pela drenagem de energia em dispositivos Android (MONTEIRO et al., 2023). A eliminação desse tipo de tarefa, previne que o dispositivo acorde da hibernação por motivos irrelevantes, maximizando a autonomia em `textitle`.

O formato JSON confere flexibilidade ao projeto por sua sintaxe de fácil leitura e modificação, permitindo ao usuário adicionar ou remover *apps* da lista se necessário com facilidade. Sob a perspectiva conceitual, essa modularidade alinha-se aos preceitos do Minimalismo Digital (NEWPORT, 2019), o qual preconiza que a tecnologia deve ser configurada de forma seletiva e intencional.

Porém, para o script ler e manipular-lo de forma efetiva, foi necessário a integração da ferramenta de linha de comando "jq", foi implementada em uma estrutura de repetição que percorre toda a lista e executa uma tentativa de desinstalação para cada objetos. Em adição, o *looping* também salva um arquivo de *log* com o resultado de cada um dos comandos, o que permite a auditoria do processo e validação da execução. A intenção é que todas as camadas salvem seus resultados no arquivo de log.

Ao término do *debloat*, a camada de aplicação já está quase completa, restam agora a instalação das ferramentas que serão essenciais no uso do produto final. Instalar ferramentas no android usando o *adb* está longe de ser uma tarefa complicada, o verdadeiro desafio é saber de onde baixar os pacotes (APKs), já que a play store por exemplo não expõe nenhum tipo de *API*. Para contornar esse desafio, optou-se por utilizar aplicativos open-source disponíveis diretamente dos releases do github, esse que possui uma *API* acessível. Do âmbito da ergonomia cognitiva do usuário, trocar o launcher padrão significa remover elementos gráficos apelativos e notificações projetadas para reter atenção.

5.2 Intervenção em Nível de Framework

Neste capítulo vamos explorar o funcionamento dos Frameworks e processos de gerenciamento do sistema Android, bem como as modificações que o Inkdroid fará no tablet

alvo nessas camadas. Entretanto, é importante deixar claro, o que estamos chamando de camada de Framework durante essa pesquisa, é na verdade um grupo de outras quatro camadas menores que serão exploradas nesta etapa, sendo elas compostas por: APIs de sistema, Android Runtime (ART), Daemons de sistema, bibliotecas nativas (em C ou C++) e o HAL

5.2.1 Conceituação e Arquitetura de Framework

Segundo (TANENBAUM; BOS, 2016), sistemas operacionais foram criados tendo como principal objetivo, serem uma forma de abstração para o desenvolvimento de software, fazendo com que a partir de sua criação, o desenvolvedor de programas comuns, não precisasse mais se preocupar com a complexidade do hardware, permitindo desenvolver sem ter um entendimento avançado de eletrônica, ou saber por exemplo, como gerenciar a voltagem de um chip de memória. Quando nos referimos aos frameworks do sistema, estamos falando sobre várias dessas abstrações organizadas, que, formam uma pilha de software (software stack) contendo APIs, bibliotecas, SDKs, Daemons e etc. além de outras peculiaridades diferentes para cada sistema.

5.2.2 Fluxo de Execução e Pipeline

Para entendermos como se dá o funcionamento dessas abstrações do sistema android, vamos usar um exemplo simples. Imagine que um desenvolvedor quer que seu aplicativo reproduza um efeito sonoro simples, se ele fizesse isso sem utilizar nenhuma abstração, ele precisaria lidar com diferenças entre vários dispositivos de som diretamente em seu código, mas usando a API do android, tudo que ele precisa fazer é dizer "Sistema operacional, reproduza o som do arquivo soundeffect.mp3 no auto-falante do sistema." essa instrução então passa por todas as camadas até chegar finalmente ao hardware e reproduzir o som de fato.

O primeiro lugar onde essa instrução passa é na Java API, aqui é onde o sistema vai validar várias informações antes de iniciar o processo pesado, fazendo perguntas como: "todos os serviços do sistema de som foram iniciados corretamente?", depois, essa requisição passa pela JNI (Java Native Interface) que vai instanciar uma classe do cliente de audio (audioserver), que é o daemon responsável por gerenciar serviços de audio (ANDROID DEVELOPERS, 2025; AOSP, 2024)

O conceito de "classe" não vem ao caso agora, mas nessa situação, você pode pensar nela como uma abstração para tarefas que o audioserver vai executar. Através dela, componentes que rodam em Java ou Kotlin podem interagir com esse processo, que roda em C++ (Linguagem usada na maioria dos Daemons do android) sendo assim, é como se cada tarefa do daemon que componentes do ART ou JVM precisam interagir, tenham sua

representação em uma classe.

Agora a situação é: o daemon já sabe da intenção do aplicativo de reproduzir um som, e a responsabilidade dele é preparar tudo que for necessário para reproduzir-lo. Uma serie de outros processos mais específicos e técnicos acontecem nesse meio tempo, então vamos focar apenas nos que são relevantes para a natureza do Inkdroid. Dentre eles, o primeiro é validação de estado do aparelho.

O daemon é um serviço de sistema e como o padrão de segurança do android é isolar os processos em sandbox, ele roda em uma parte do sistema chamada *Daemon Space* que não consegue acessar o chamado *User Space*, que é o local onde as informações de estado que ele quer estão. Por conta desse isolamento, o audioserver precisa fazer uma requisição IPC via Binder, o barramento nativo de IPC do android, que empacota e envia a mensagem para o *AudioService*, que é o módulo de audio que roda no *User Space*. (AOSP, 2024)

O *AudioService* é quem controla configurações como o volume que o usuário definiu, modo silencioso, sabe qual aplicativo tem o "Audio Focus", quem tem permissão para reproduzir audio agora e etc. O audioserver recebe a resposta e imediatamente passa ela pelas suas próprias políticas e regras, que comparam as informações do *User Space* com estado dos conectores e determina a rota que os dados vão fazer até o auto-falante.

Depois dessa validação, vem a preparação do dado. No exemplo atual, isso significa preparar o arquivo mp3, que usa técnicas de compressão de dados, por isso não pode ser reproduzido diretamente e precisa de uma decodificação. isso é feito através do motor de midia Stagefright/Codec2, assim transformando o arquivo mp3 em um PCM (Pulse Code Modulation), que não é nada mais que um formato para armazenar o som, ou melhor, a posição da membrana do auto-falante para cada milissegundo do audio. É o movimento dessa membrana que cria as ondas sonoras.

Entretanto, o PCM gerado ainda não é aquilo que vai ser reproduzido, o daemon ainda vai usar o *AudioFlinger* para fazer a mixagem, ou seja, juntar todos os sons que vão ser reproduzidos na mesma faixa de tempo, em um único PCM.

O PCM resultante desce mais um nível até o HAL, mais especificamente no módulo de audio do HAL que é fornecido pelo fabricante do SoC. ele traduz todas as instruções das camadas mais altas, em instruções específicas para o hardware, nesse caso, o chip de audio do SoC. ele vai configurar a rota fornecida pelo audioserver para o a saída de audio e fazer system calls para o driver de audio do Linux, o ALSA (Advanced Linux Sound Architecture), que agora sim pode se comunicar diretamente com o hardware, que nesse caso é o DAC (Digital to Analog Converter), que converte o sinal digital do PCM (zeros e uns) em sinal analógico (Níveis variaveis de energia). Esses sinais são enviados para o auto-falante que finalmente reproduz o som, tudo isso quase instantaneamente.

(TANENBAUM; BOS, 2016; ANDROID DEVELOPERS, 2025)

O objetivo desse exemplo é entender o papel de cada um dos componentes de software citados e como eles se integram entre si, além de saber que para cada peça do sistema, existe um pipeline essencialmente idêntico a esse, como por exemplo renderização gráfica, rede internet, bluetooth ou até sinal de celular.

5.2.3 Estratégia de Modificação não Invasiva

Com base na explicação da sessão anterior, e nas observações que Yaghmour (2013) faz em seu livro, podemos notar que a camada de framework é extremamente complexa, delicada e interconectada além de ser peça central do sistema, sendo assim, modificar suas bibliotecas e executáveis compilados pode induzir inúmeros softbricks. Por conta disso, o projeto foi levado a encontrar outra maneira segura de modificar a natureza do funcionamento dos componentes da camada de framework, sem tornar isso um trabalho arduo e perigoso de engenharia reversa, o que levou a ideia de modificar diretamente as regras de negócio e políticas que regem os componentes de software do framework android, mudando o resultado das diferentes validações do sistema através de configurações nativas que ficam muitas vezes ocultas.

Segundo a documentação oficial (ANDROID DEVELOPERS, 2025; AOSP, 2024) o sistema android guarda suas políticas e configurações em um banco de dados relacional, feito em SQLite que armazena grande parte das configurações modificáveis do sistema android, em especial da camada de framework e do User Space. o ADB tem comandos específicos para ler e modificar essas configurações, respectivamente: “adb settings get” e “adb settings put”. Esse também é o mesmo comando para trocar o launcher padrão, usado anteriormente, trocando o valor da da célula que dizia qual app abrir ao usar o botão home.

Além do settings db, componentes do sistema como Window Manager e SurfaceFlinger possuem suas próprias políticas e comandos para modificar-las.

5.2.4 Display e Síndrome da Visão do Computador

Sendo a tela o ponto chave de venda, uso e desejo por e-readers dedicados, nesse projeto ela não poderia deixar de ser um ponto de pesquisa e modificação. Entretanto, primeiro precisamos entender o que faz do E-ink uma opção ergonomicamente melhor para longas leituras, e no que as telas LCD pecam, para que possamos tentar resolver esses problemas. No início de seu desenvolvimento, o E-ink tinha como objetivo ser uma tela digital de baixo consumo energético e que se comportasse visualmente como papel, ela não foi pensada para ser ergonômica ou para mitigar o CVS, isso na verdade foi uma consequência descoberta apenas depois, e se tornou o centro do marketing das empresas

que a comercializam. A Associação Americana de Optometria (AOA) define a síndrome da visão do computador como um conjunto de problemas oculares resultantes do uso de telas. Estima-se que a prevalência da CVS entre usuários de tela, ultrapassa números de 70% mundialmente (Sá, 2016)

No estudo de Benedetto et al. (2013), é feita uma comparação entre telas LCD e E-ink, que provam com testes que o número de piscadas por minuto diminui ao usar telas LCD. Sá (2016) vai além e quantifica essa queda, descobrindo que nas telas LCD, uma pessoa pisca de 5 a 7 vezes durante uma tarefa de concentração como leitura, enquanto em um papel comum a média é de 22 vezes por minuto, isso tem como consequência uma maior evaporação das lágrimas, dando a sensação de olho seco e cansado.

Em adição, outro agente popularmente conhecido de fadiga e cansaço ocular é a luz azul, porém ao contrário da crença popular, a luz azul não está diretamente ligada a CVS ou a desconfortos visuais (que são causados na verdade pela exposição a todo tipo de luz artificial de telas quando não há um número normal de piscadas), mas sim ao sono. "A temperatura de cor e a intensidade da luz influenciam diretamente os processos psicológicos não visuais e o ritmo circadiano" (KNEZ, 2001). A exposição desenfreada a luz azul portanto, inibe a secreção natural de melatonina, gerando um efeito semelhante a cafeína, afetando a qualidade do sono e gerando estresse psicológico

O que as telas E-ink fazem então para mitigar esses problemas é emitir pouca ou nenhuma luz. Sua tela reflete a luz do ambiente assim como o papel, e sua natureza estática de pouco movimento diminui o estresse dos olhos. Essas características não são possíveis de se reproduzir fielmente em uma tela LCD comum, o que nos obriga a encontrar outros métodos que ajudem a resolver os problemas citados. Em seu estudo, Benedetto et al. (2013) nos diz exatamente o que precisamos, ele afirma que a fadiga visual em painéis LCDs pode ser mitigada via software, através do ajuste de brilho, aumento do contraste do texto, polaridade e redução da saturação. São essas as principais alterações que o InkDroid aplica ao tablet usando as configurações do settings.db, e vamos passar por uma de cada vez agora.

1. **Movimentos e Animações:** Um dos pontos antes citados para o E-ink ser tão ergonômico é a falta de movimentos e animações. O android é um sistema que está cheio de transições e animações complexas, e existem inúmeros motivos para retirarmos elas, além da própria ergonomia. As animações foram colocadas no sistema para passar a ideia de fluidez, mas em específico no tablet alvo dessa pesquisa, elas acabam causando maior overhead, resultando em travamentos ao renderizar-las, além de serem consideradas agentes de distração. Felizmente, o projeto android (ANDROID DEVELOPERS, 2025) documenta configurações relacionadas a animações do sistema, sendo possível desativar-las completamente modificando a tabela "global" no settings.db,

mais especificamente definindo como 0 as opções "window_animation_scale", "transition_animation_scale" e "animator_duration_scale". Outra alteração relacionada ao movimento da tela é a taxa de atualização. seria incrível se fosse possível diminuir a taxa de atualização sem limites, porém cada tela tem seus limites e presets, por isso, o que o script de instalação faz nesse sentido é colocar a menor taxa disponível, que no tablet alvo da pesquisa é de 30hz. Isso não tem grande impacto na ergonomia mas ajuda bastante na autonomia da bateria

2. **Cores e Conforto Visual:** Tendo agora um nível consideravelmente menor de movimento, os agentes de desconforto visual se concentram nas cores e intensidade da luz. para mitigar isso o Inkdroid disponibiliza dois perfis de configuração, o monocromático, que usa um modo específico para pessoas daltonicas para diminuir consideravelmente o números de cores e estímulos visuais, e um modo que ativa apenas o filtro de luz azul. Essas além desses dois perfis de configuração, a instalação também faz alterações na densidade de pixel e no contraste durante a leitura de texto. Essas alterações são feitas de maneira idêntica as feitas no tópico anterior, com a exceção de que agora são feitas na tabela "secure" e em outros parametros.

Essas, até então, são todas as alterações possíveis de se fazer de maneira segura e não invasiva na tela. Não chega a ser tão confortável quanto uma tela E-ink, mas tem o potencial de melhorar consideravelmente os efeitos de leituras longas se usado em baixo brilho, além de ser esteticamente mais proxima e diminuir consideravelmente travamentos e uso de bateria. Seria possível limitar o brilho do aparelho também, mas durante o desenvolvimento, essa parte foi entendida como algo que o usuário precisa ter controle e ajustar sozinho de acordo com sua preferência pessoal.

5.3 Bateria, Autonomia e Foco

Depois da tela, a proxima coisa que mais gera desejo por um leitor dedicado, é a longa duração da bateria e a ausência de distrações. Muito dessa autonomia, que pode vir a durar dias, vem da própria tela, já que ela é extremamente eficiente, mas quando prestamos um olhar mais atento na comparação com um tablet android comum, vemos inumeras coisas que o tablet tem e faz e que os leitores focados não. Essas coisas podem ser responsáveis por drenar a bateria, já que tudo o que demanda processamento, demanda energia. Além disso, todas as funções disputam a atenção do usuário igualmente

A primeira coisa portanto que o Inkdroid faz nesse sentido é ativar a opção "always_finish_activities". Como descrito em seu próprio nome, o que ela faz é impedir que qualquer aplicação rode em segundo plano, diminuindo portanto o uso de memória RAM e processamento. A maioria das alterações aqui vão seguir a mesma natureza, então, ao invés de passar por uma de cada vez, vamos agrupar-las.

1. **Modo de Economia Permanente:** Como o sistema android não disponibiliza uma forma de underclock ou de travar de forma permanente o modo de economia de bateria, ao invés disso, o que podemos fazer é simplesmente mudar de 15% para 100% a ativação automática, mudando a configuração "low_power_trigger_level". Isso garante que a economia de energia seja o padrão, e liga ela novamente sempre que o dispositivo reiniciar, apesar disso o usuário ainda poderá desativar-la facilmente.
2. **Conectividade Sem Fio:** A internet e bluetooth são coisas extremamente convenientes no uso de qualquer aparelho, inclusive para um leitor, entretanto o android possui uma peculiaridade de estar sempre buscando por essas conexões em segundo plano. Mudando as opções "wifi_scan_always_enable" e "ble_scan_always_enable" para 0, essas buscas em segundo plano são desativadas, e só acionadas quando o usuário as chama. Somado a isso, também desativamos a opção "auto_sync", que impede que aplicativos sincronizem contas em segundo plano, e como configuração opcional, durante a instalação, o Inkdroid pergunta ao usuário se ele quer desativar a rede mobile de dados móveis "mobile_data" e manter apenas o Wi-Fi, já que o projeto tem por intenção de ser usado em tablets "abandonados", interpreta-se que esses aparelhos não possuam chip SIM.
3. **Sons, Notificações e Vibrações:** O motor de vibração dos dispositivos móveis é algo que nem sempre é levado em consideração quando se fala de autonomia de bateria, mas nesse projeto não tem por que manter ativado. Mesmo que a redução da bateria seja mínima ela ainda existe, e ele ainda pode distrair o usuário de diferentes formas, e, particularmente, não gostaria que meu E-reader vibrasse o tempo todo. Podemos desativar-lo na opção "haptic_feedback_enabled", isso desativa as vibrações do sistema, mas outros apps ainda poderiam acionar o haptic feedback, então o que podemos fazer é diminuir a intensidade ("haptic_feedback_intensity") para 0. Além disso, as notificações de push (heads_up_notifications_enabled) também é desativado junto com a ativação do "zen_mode", ou não pertube. Efeitos sonoros meramente estéticos também perdem o espaço durante a instalação.
4. **Configurações Extra:** Essas são modificações que julgo difícil de encaixar em uma única categoria, mas pequenas demais para terem sua própria. Primeiro, o "screen_off_timeout", que define o desligamento automático da tela. Nesse caso, foi definido como 1800000 segundos (30 minutos). A segunda alteração é referente ao assistente de voz "Ok Google", que, está a 100% do tempo perguntando "será que alguém me chamou?" e consumindo processamento. Talvez você tenha interpretado isso como uma simplificação, mas na verdade isso é algo que acontece exatamente desse jeito. Em casos extremos como nos celulares IOS, um chip especializado é responsável por processar entrada de áudio o tempo inteiro, sempre perguntando "Alguém disse 'Eai Siri' no ultimo segundo?". No android entretanto, isso é feito por

meio de software com um daemon, mas que ainda sim possui impacto ativo no uso de processamento e memória. Por fim, também desativamos atualizações OTA (Over The Air), para prevenir que elas quebrem ou retirem todas as alterações que fizemos até então

Até o momento, essas são todas as alterações feitas sobre a camada de framework. Mais coisas podem vir a ser adicionadas no futuro, mas acredita-se que com essas configurações, possamos tornar o tablet alvo mais minimalista, e conveniente para seu usuário

5.4 Lógica de Programação, Arquitetura e Ferramentas do InkDroid

Tendo analisado a arquitetura do sistema e as alterações no nível de *framework* na seção anterior, detalham-se aqui a estrutura lógica do código-fonte, as rotinas de automação em *shell script* e o funcionamento das principais ferramentas desenvolvidas na CLI do InkDroid.

5.4.1 Evolução do Script, Modularidade e Modos de Execução

O desenvolvimento da ferramenta de instalação do InkDroid evoluiu para uma arquitetura modular orientada a funções em Bash, o que facilita o reaproveitamento de rotinas, a manutenção e a extensibilidade do código. A automação suporta execução interativa e não interativa por meio de interpretação de argumentos de linha de comando (*CLI flags*). Através da função `parseArguments()`, processam-se etapas específicas (como `--only-debloat`, `--only-apps` ou `--only-display`), a omissão de dependências (`--skip-deps`) e a confirmação automática (`-y` ou `--yes`). O controle do fluxo de execução é orquestrado pela função `main()`, que invoca condicionalmente cada subrotina de acordo com os parâmetros fornecidos.

5.4.2 Verificação de Segurança e Preparação do Ambiente

O fluxo de execução inicia-se pela função `installDependencies()`, que identifica o gerenciador de pacotes do sistema hospedeiro Linux (`apt-get`, `dnf`, `pacman` ou `zypper`) e garante a presença de ferramentas essenciais de manipulação, como `android-tools`, `jq`, `curl` e `bc`, isso garante que todos os componentes necessários para a execução estejam instalados, evitando assim uma interrupção inesperada. Em seguida, a função `checkDevice()` consulta os dispositivos via comunicação ADB e contabiliza os aparelhos detectados em modo de depuração USB, interrompendo a execução de comandos caso nenhum ou múltiplos dispositivos sejam encontrados de forma simultânea. Quando identificado um único aparelho compatível, a função extrai propriedades sistêmicas (`ro.product.manufacturer`, `ro.product.model`, `ro.build.version.release`) e invoca a rotina `confirmDevice()`

para solicitar a validação de segurança por parte do usuário, garantido sempre que o usuário saiba o que esta acontecendo na execução e aonde.

5.4.3 Inicialização de Logs, Leitura da Blacklist e Debloat Automático

Validada a conexão, a rotina `initLog()` verifica a configuração no arquivo `blacklist.json` e cria o documento de auditoria `debloatlog.txt`, que registra no cabeçalho o modelo do dispositivo e a data de execução, assim em caso de erros, pode-se verificar o que aconteceu. O fluxo de desinstalação apoia-se em duas funções sequenciais. A primeira, `parseBlacklist()`, extrai os identificadores de pacotes utilizando `jq` ou um interpretador Python 3 como segunda opção (*fallback*). A segunda, `debloatPackages()`, processa essa lista em um ciclo de repetição, executando a seguinte instrução: `adb shell pm uninstall -k --user 0 "$pacote"`. Esta lógica garante que os aplicativos sejam desinstalados exclusivamente no espaço do usuário principal, processando e registrando em log os retornos do sistema (`Success`, `not installed` ou falhas) em tempo real.

5.4.4 Download, Instalação de Aplicações e Definição do Launcher Padrão

O processo de instalação de dependências externas apoia-se primariamente na função `getReleaseApkUrl()`, que realiza requisições à API REST do GitHub via `curl` e processa os metadados JSON para extrair dinamicamente a URL do ativo (*asset*) APK mais recente. As funções de implantação gerenciam o pacote da seguinte forma:

Os metadados são interpretados para extrair exatamente a versão mais recente dos dois apps, o que permite o download e transferência para `/tmp/Koreader.apk`, Um diretório de arquivos temporários no computador, onde ele fica temporariamente até ser instalado por meio da instrução `adb install -r --user 0`.

Para isolar o ambiente, a função `setDefaultLauncher()` aplica o comando `adb shell cmd role add-role-holder android.app.role.HOME app.olauncher`, forçando a substituição da interface padrão via RoleManager do Android. Esse passo restringe notificações flutuantes e estímulos gráficos, baseando-se na filosofia do Minimalismo Digital (NEWPORT, 2019).

5.4.5 Otimizações Ergonômicas do Display via Código

Para implementar os perfis de mitigação visual estabelecidos anteriormente, o script utiliza a função `configureDisplay()`, que automatiza as injeções de parâmetros no banco de dados `settings.db`. A supressão das animações globais do sistema é realizada forçando os valores `window_animation_scale`, `transition_animation_scale` e `animator_duration_scale` para zero. Em seguida, o algoritmo processa a saída do serviço nativo via `dumpsys display` para isolar a menor taxa de atualização suportada

pelo *hardware*, injetando limitadores correspondentes nas chaves `peak_refresh_rate` e `min_refresh_rate`. A execução dos perfis ocorre por meio dos seguintes ajustes:

- **Perfil Monocromático:** Manipula variáveis de acessibilidade injetando `accessibility_display` 1 e definindo o índice de escala de cor com `accessibility_display_daltonizer` 0.
- **Filtro de Luz Azul:** Altera a emissão do painel configurando `night_display_activated` 1 e estabelecendo a temperatura de cor em `night_display_color_temperature` 3000.

5.4.6 Ajustes Sistêmicos de Bateria e Funcionalidades da CLI

A implementação das políticas de economia energética e restrição de conexões é orquestrada pela função `apply_Framework_Tweaks()`. A subrotina processa a injeção em lote diretamente nas tabelas `global`, `secure` e `system` do `settings.db`. É este módulo do código que ativa o encerramento de atividades em segundo plano (`always_finish_activities` 1), eleva permanentemente o limitador de economia (`low_power_trigger_level` 100), estende o tempo de bloqueio de tela para trinta minutos (`screen_off_timeout` 1800000) e desativa chaves de rastreamento contínuo (`wifi_scan_always_enabled`, `haptic_feedback_enabled`, `zen_mode`).

O projeto disponibiliza adicionalmente a interface de linha de comando `ink`, projetada para operar como um utilitário CLI interativo global com os seguintes subcomandos estendidos:

- `ink status`: Aciona a função `cmdStatus()` para consultar dinamicamente propriedades sistêmicas via `getprop`, capturar a bateria via `dumpsys battery`, e extrair a densidade e resolução de tela com `wm density` e `wm size`.
- `ink push <arquivo>`: Executa a rotina `cmdPush()` para transferir e-books (.epub ou .pdf) para o diretório `/sdcard/Books/` via `adb push`. Após a transferência, o script emite programaticamente um sinal (*broadcast*) ao `MEDIA_SCANNER` do Android, forçando a atualização indexada do `KOReader`.
- `ink display <profile>`: Invoca a função `cmdDisplay()`, processando argumentos textuais no terminal para aplicar, em tempo real, os modos `monochrome`, `blue` ou realizar o `reset` da exibição.

5.4.7 Arquitetura de Reversibilidade (`InkDroid_uninstall.sh`)

Para conformidade com os princípios de governança, o script de desinstalação reverte o estado do sistema ao padrão de fábrica do ecossistema do dispositivo, sem

necessariamente formata-lo, o que faria o usuário perder seus arquivos. O processo lê os mesmos arquivos de configuração JSON para reabilitar os pacotes previamente desativados via:

```
adb shell cmd package install-existing <package_name>
```

Restaura também os parâmetros de densidade de tela originais e remove as aplicações e modificações introduzidas pelo projeto.

O Utilitário fetchbook E Resiliência de Rede

O utilitário **fetchbook** atua como a camada de aquisição de conteúdo do Inkdroid. Projetado originalmente como um simples cliente de busca para repositórios abertos, o script passou por uma reformulação arquitetural profunda devido a transformações na infraestrutura da internet.

5.4.8 Necessidade do Refatoramento de Rede

A primeira versão do utilitário enfrentava degradação operacional severa causada por dois fatores principais:

1. **Apreensão de Domínios (*Domain Seizures*):** A instabilidade de domínios públicos de bibliotecas abertas causava falhas imediatas de resolução de DNS.
2. **Bloqueios da Cloudflare e Desafios JS/CAPTCHA:** A implementação massiva de proteções contra bots e requisições automatizadas inviabilizou a raspagem de dados (*scraping*) via requisições HTTP diretas por `curl`.

A Tabela 1 resume a transição de arquitetura implementada na ferramenta para garantir alta disponibilidade:

Tabela 1 – Comparativo Arquitetural entre a Versão Legada e a Nova Abordagem Multicamada

Vetor de Análise	Arquitetura Legada	Nova Arquitetura Multicamada
Resolução de Fontes	Endpoints HTTP fixos / Específicos únicos	Provedores encadeados com fallback automático
Parsing de Dados	Expressões regulares (<code>grep/sed</code>) em HTML	Parsing JSON nativo e estruturado via Python
Distribuição de Conteúdo	Download HTTP direto com dependência de host	Redes descentralizadas (Gateways IPFS)
Tratamento de Desafios	Falha genérica em bloqueios de borda	Redirecionamento a APIs públicas e protocolos P2P

Arquitetura Multicamada de Busca

Para assegurar o provimento contínuo de dados sem interrupções, o novo **fetchbook** estrutura a busca em três níveis hierárquicos de alta resiliência:

1. **Camada Primária (Project Gutenberg API):** Prioriza obras em domínio público com metadados estruturados e garantias de estabilidade de servidor.
2. **Camada Secundária (Open Library):** Atua como ponte para catalogação internacional e identificação de metadados via ISBN/OLID.
3. **Camada Terciária e Fallback Descentralizado (LibGen / Gateways IPFS):** Em caso de indisponibilidade das camadas primárias ou de títulos específicos, o sistema recorre ao ecossistema LibGen. Se os espelhos HTTP tradicionais responderem com bloqueios de borda (e.g., código 403 **Forbidden** ou desafio Cloudflare), o script extrai o hash IPFS do documento e efetua o download através de múltiplos gateways IPFS públicos (e.g., `ipfs.io`, `dweb.link`, `cloudflare-ipfs.com`).

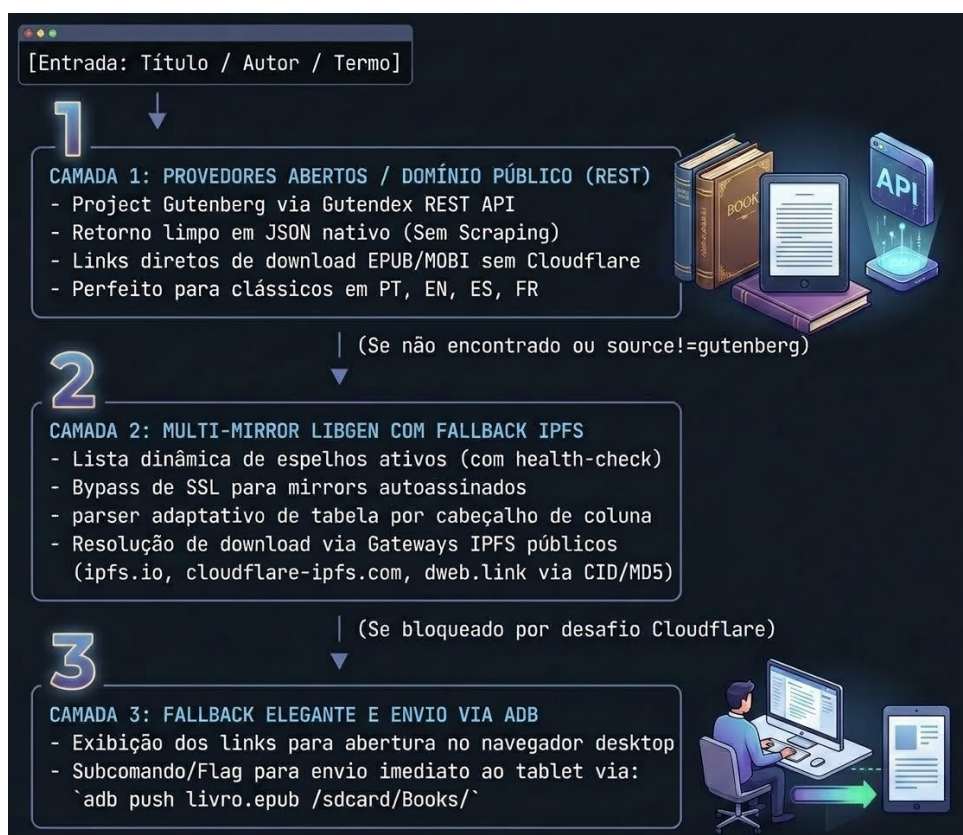


Figura 1 – A Figura ilustra o fluxo condicional de resiliência e fallback executado pelo utilitário **fetchbook**.

5.4.9 Integração Híbrida Bash-Python

Embora a estrutura principal do `fetchbook` permaneça em Bash para manter a compatibilidade com o ambiente de terminal Unix da ferramenta `ink`, a manipulação de dados complexos foi transferida para um interpretador Python inline.

O script executa blocos Python via `python3 -c` para realizar o parsing das estruturas JSON retornadas pelas APIs REST. Esta abordagem elimina a fragilidade de expressões regulares no tratamento de strings codificadas em UTF-8, sanitiza os títulos dos livros para evitar injeção de comandos no shell e gerencia a lógica de seleção de Gateways IPFS quando uma falha HTTP é detectada.

Transferência Automatizada para o Dispositivo (`pushToDevice`)

Após a conclusão bem-sucedida do download e validação da integridade do arquivo `.epub` ou `.pdf`, o script aciona a função `pushToDevice`:

1. Executa a verificação de estado do daemon ADB.
2. Mapeia o diretório de destino configurado no dispositivo (por padrão, `/sdcard/Books/` ou o diretório de biblioteca do KOREader).
3. Transfere o arquivo via `adb push <local_file_path> <remote_directory>`.
4. Emite um sinal de atualização do armazenamento de mídia (`android.intent.action.MEDIA_SC`) para que a nova obra seja indexada imediatamente pela biblioteca do leitor digital sem a necessidade de reiniciar o aplicativo de leitura, caso aberto.

6 RESULTADOS

7 CONSIDERAÇÕES FINAIS

Espera-se demonstrar que os dispositivos legados ainda possuem alto potencial técnico quando desonerados de softwares mal otimizados, podendo ser adaptados para tarefas que não demandam alto desempenho, assim evitando o descarte. O artefato final servirá como prova de conceito para políticas de TI sustentáveis, provando que a engenharia de software consciente pode converter o que seria resíduo eletrônico em um ativo educacional valioso e focado no ser humano.

8 AGRADECIMENTOS

Referências Bibliográficas

ABREE. *Relatório de Resultados de Logística Reversa de Eletroeletrônicos e suas Embalagens 2023-2024*. São Paulo, 2024. 11

ANDROID DEVELOPERS. *Android Debug Bridge (adb)*. 2025. Disponível em: <https://developer.android.com/tools/adb>. Acesso em: 1 ago. 2026. 12, 14, 17, 19, 20

AOSP. *System Architecture and Platform Documentation*. 2024. Disponível em: <https://source.android.com>. Acesso em: 15 mar. 2026. 17, 18, 19

BALDÉ, C. P. et al. *The Global E-waste Monitor 2024: Quantities, flows and the global e-waste challenge*. Geneva/Bonn, 2024. 9, 11

BENEDETTO, S. et al. E-readers and visual fatigue: Comparison between e-ink and lcd. *PLOS ONE*, v. 8, n. 12, p. e83676, 2013. 12, 20

KNEZ, I. Effects of colour of light on non-visual psychological processes. *Journal of Environmental Psychology*, v. 21, n. 2, p. 201–208, 2001. 20

MONTEIRO, E. et al. Analysis of energy consumption on android devices for developers: A systematic mapping study. *SBC Reviews on Computer Science*, v. 29, n. 1, 2023. 10, 16

NEWPORT, C. *Minimalismo Digital: para uma vida profunda em um mundo superficial*. Rio de Janeiro: Alta Books, 2019. 11, 16, 24

PARNAS, D. L. Software aging. In: *Proceedings of the 16th International Conference on Software Engineering*. Hamilton: IEEE, 1994. p. 279–287. 9, 15

PONTES, J. C. N. *Análise de desempenho e consumo de energia de parâmetros TCP/IP em dispositivos móveis*. 71 p. Dissertação (Mestrado) — Centro de Informática, Universidade Federal de Pernambuco, Recife, 2017. 13

Sá, E. C. *Síndrome da visão do computador e função visual em trabalhadores usuários de computador de um hospital público universitário de São Paulo: prevalência e fatores associados*. 107 p. Tese (Doutorado) — Faculdade de Saúde Pública, Universidade de São Paulo, São Paulo, 2016. 9, 12, 20

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. 14, 17, 19

YAGHMOUR, K. *Embedded Android: Porting, Extending, and Customizing*. 1. ed. Sebastopol: O'Reilly Media, 2013. 19